

Homework 1: Advanced Python for Data Science

Shravan Khunti
ssk10036@nyu.edu
NetID: ssk10036

Spring 2026
DS-GA 1019

1 Problem 1: Counting Representations (30 points)

1.1 Problem Statement

Write a function $f(n)$ to calculate the number of ways of representing n as a sum of 1, 2, and 5, where the order of summands is important.

1.2 Solution Approach

The problem can be modeled as a recurrence relation:

$$f(n) = f(n-1) + f(n-2) + f(n-5) \quad (1)$$

with base cases:

$$\begin{aligned} f(n) &= 0 \quad \text{for } n < 0 \\ f(0) &= 1 \end{aligned}$$

1.3 Part 1: Recursive Solution

A direct implementation of the recurrence relation.

n	Time (seconds)
10	0.000029
25	0.074646

Table 1: Timing results for recursive solution

1.4 Part 2: Memoized Solution

The recursive solution with a dictionary to cache results for previously computed values of n . This significantly reduces the time complexity from exponential to linear.

1.5 Part 3: Iterative Solution

A dynamic programming approach using an array (or list) to build the solution from the bottom up $(f(0), f(1), \dots, f(n))$.

n	Time (seconds)
10	0.000007
25	0.000011
50	0.000021
100	0.000113

Table 2: Timing results for memoized solution

n	Time (seconds)
10	0.000005
25	0.000005
50	0.000009
100	0.000019

Table 3: Timing results for iterative solution

2 Problem 2: N-Body Simulation Optimization (70 points)

2.1 Problem Statement

Optimize an N-body simulation code (`nbody.py`) to run significantly faster with a target execution time of less than 30 seconds.

2.2 Optimization Strategy

I implemented four versions of the code, addressing specific areas of improvement as requested.

2.2.1 Version 1 (`nbody_1.py`): Reduced Function Call Overhead

- Inlined the helper functions (`compute_deltas`, `compute_b`, `compute_mag`, `update_vs`, `update_rs`, `compute_energy`) directly into the main `advance` function.
- This removes the overhead of pushing/popping stack frames for millions of small function calls.
- **Total time:** ~ 11.3 seconds

2.2.2 Version 2 (`nbody_2.py`): Alternatives to Membership Testing

- The original code used a list `seenit` and checked `if body2 in seenit` inside a nested loop, which is $O(N)$ inside $O(N^2)$, making it inefficient.
- Replaced this with an index-based double loop `for i in range(n): for j in range(i+1, n):`, which ensures each pair is visited exactly once without any list lookups.
- **Total time:** ~ 6.9 seconds

2.2.3 Version 3 (`nbody_3.py`): Using Local Variables

- The original code accessed the global dictionary `BODIES` repeatedly inside the inner loops.

- In Python, accessing global variables is slower than accessing local variables. Created a local list `bodies = list(BODIES.values())` and passed it to the functions.
- This version proved to be the fastest optimization.
- **Total time:** ~ 5.8 seconds

2.2.4 Version 4 (`nbody_4.py`): Using Data Aggregation

- Flattened the data structure into single lists for positions (r), velocities (v), and masses (m) to avoid the overhead of unpacking tuples.
- However, in pure Python, the overhead of list indexing and lack of vectorization (unlike NumPy) meant this approach was slightly slower than `nbody_3`.
- **Total time:** ~ 6.1 seconds

2.3 Final Optimization (`nbody_opt.py`)

The final version incorporates the most effective optimizations:

- Uses the local variable strategy from `nbody_3.py`
- Uses the optimized loop structure from `nbody_2.py`
- Keeps the code clean and readable while maximizing Python's interpreter efficiency

2.4 Speedup Analysis

Metric	Value
Original Time (100 loops)	27.57 seconds
Optimized Time (100 loops)	9.40 seconds
Speedup Factor	$2.93\times$

Table 4: Performance comparison between original and optimized versions

3 cProfile Logs (Problem 2)

Below are the cumulative time profiles for each version of the N-body simulation.

3.1 nbody_1.py Profile Analysis

- Shows high number of function calls removed compared to original (implied), but still significant time in the advance loop.
- Total time: ~11.3 seconds

36003706 function calls in 11.310 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	11.310	11.310	{built-in method builtins.exec}
1	0.000	0.000	11.310	11.310	nbody_1.py:1(<module>)
1	0.249	0.249	11.310	11.310	nbody_1.py:137(nbody)
2000000	9.253	0.000	11.060	0.000	nbody_1.py:44(advance)
20002000	1.069	0.000	1.069	0.000	{method 'append' of 'list'}
14001401	0.738	0.000	0.738	0.000	{method 'keys' of 'dict'}
200	0.001	0.000	0.001	0.000	nbody_1.py:92(report_energy)

3.2 nbody_2.py Profile Analysis

- Significant reduction in time due to optimized loop structure (removing seenit checks).
- Total time: ~6.9 seconds

8000906 function calls in 6.918 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	6.918	6.918	{built-in method builtins.exec}
1	0.000	0.000	6.918	6.918	nbody_2.py:1(<module>)
1	0.243	0.243	6.918	6.918	nbody_2.py:126(nbody)
2000000	6.393	0.000	6.674	0.000	nbody_2.py:44(advance)
4000401	0.215	0.000	0.215	0.000	{method 'keys' of 'dict'}
2000200	0.065	0.000	0.065	0.000	{built-in method builtins.len}

3.3 nbody_3.py Profile Analysis

- Further reduction by using local variables (bodies list) instead of global dictionary lookups.
- This was the fastest version.
- Total time: ~5.8 seconds

4000507 function calls in 5.844 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	5.844	5.844	{built-in method builtins.exec}
1	0.000	0.000	5.844	5.844	nbody_3.py:1(<module>)
1	0.221	0.221	5.844	5.844	nbody_3.py:123(nbody)
2000000	5.559	0.000	5.622	0.000	nbody_3.py:44(advance)
2000200	0.063	0.000	0.063	0.000	{built-in method builtins.len}

3.4 nbody_4.py Profile Analysis

- Data aggregation (flattening lists). The overhead of list management in pure Python made this slower than `nbody_3`.
- Total time: ~ 6.1 seconds

2000323 function calls in 6.100 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	6.100	6.100	{built-in method builtins.exec}
1	0.000	0.000	6.099	6.099	nbody_4.py:1(<module>)
1	0.193	0.193	6.099	6.099	nbody_4.py:116(nbody)
2000000	5.906	0.000	5.906	0.000	nbody_4.py:44(advance)